Module#class_eval is very different from BasicObject#instance_eval, which you learned about earlier in *instance_eval()*, on page 85. instance_eval only changes self, while class_eval changes both self and the current class.

(This is not the whole truth: instance_eval also changes the current class, but you'll have to wait for *Singleton Classes and instance_eval()*, on page 127, to learn how exactly. For now, you can safely ignore the problem and assume that instance_eval only changes self.)

By changing the current class, class_eval effectively reopens the class, just like the class keyword does.

Module#class_eval is actually more flexible than class. You can use class_eval on any variable that references the class, while class requires a constant. Also, class opens a new scope, losing sight of the current bindings, while class_eval has a *Flat Scope (83)*. As you learned in *Scope Gates*, on page 81, this means you can reference variables from the outer scope in a class_eval block.

Finally, just like instance_eval has a twin method called instance_exec, module_eval/class_eval also has an equivalent module_exec/class_exec method that can pass extra parameters to the block.

Now that you know about both instance_eval and class_eval, you might wonder which of the two you should use. In most cases the answer is easy: you use instance_eval to open an object that is not a class, and class_eval to open a class definition and define methods with def. But what if you want to open an object that happens to be a class (or module) to do something other than using def? Should you use instance_eval or class_eval then?

If all you want is to change self, then both instance_eval and class_eval will do the job nicely. However, you should pick the method that best communicates your intentions. If you're thinking, "I want to open this object, and I don't particularly care that it's a class," then instance_eval is fine. If you're thinking, "I want an *Open Class (14)* here," then class_eval is almost certainly a better match.

That was a lot of information about the current class and how to deal with it. Let's recap the important points that we just went through.

Current Class Wrap-up

You learned a few things about class definitions:

- The Ruby interpreter always keeps a reference to the *current class* (or module). All methods defined with def become instance methods of the current class.